

Machine Learning

Complete Interview Preparation Guide

TABLE OF CONTENTS

- 1 What is Machine Learning?
- 2 Types of Machine Learning
- 3 Supervised vs Unsupervised Learning (Deep Dive)
- 4 Classification vs Regression (Deep Dive)
- 5 Key ML Algorithms & When to Use Them
- 6 The ML Pipeline — Step by Step
- 7 Key ML Terms & Definitions
- 8 Common Interview Questions & Answers

1 What is Machine Learning?

□ Definition

Machine Learning (ML) is a subset of Artificial Intelligence (AI) that gives computers the ability to **learn from data** and **improve from experience** without being explicitly programmed. Instead of writing rules manually, ML algorithms discover patterns from data.

□ Interview Tip — How to define ML

"Machine Learning is the science of getting computers to act without being explicitly programmed. It uses statistical methods to enable machines to improve with experience." — Arthur Samuel (1959)

Traditional Programming vs Machine Learning

Aspect	Traditional Programming	Machine Learning
Input	Data + Rules	Data + Output (Labels)
Output	Answers	Rules / Model
Approach	Hard-coded logic	Learns patterns from data
Maintenance	Manual rule updates	Retrain with new data
Example	Spam filter with keyword list	Spam filter trained on email data

AI vs ML vs Deep Learning — The Hierarchy

Artificial Intelligence (AI) — The broadest concept; machines simulating human intelligence.

↳ **Machine Learning (ML)** — A subset of AI; algorithms that learn from data.

↳ **Deep Learning (DL)** — A subset of ML; uses neural networks with many layers.

2 Types of Machine Learning

Machine Learning is broadly categorized into **four types**:

1. Supervised Learning **SUPERVISED**

What: The model learns from **labeled data** — each input has a known correct output (label).

Goal: Learn a mapping function $f(X) \rightarrow Y$ from input X to output Y .

Analogy: Like a student learning with an answer key — the teacher provides correct answers, and the student learns to generalize.

Sub-types: Classification & Regression

Examples: Email spam detection, house price prediction, disease diagnosis, credit scoring.

Algorithms: Linear Regression, Logistic Regression, Decision Trees, Random Forest, SVM, KNN, Neural Networks, Gradient Boosting (XGBoost, LightGBM).

2. Unsupervised Learning **UNSUPERVISED**

What: The model learns from **unlabeled data** — no correct output is provided.

Goal: Discover hidden patterns, structures, or groupings in data.

Analogy: Like a child sorting candies by color without being told the color names — finding structure on its own.

Sub-types: Clustering, Dimensionality Reduction, Association Rule Learning, Anomaly Detection.

Examples: Customer segmentation, topic modeling, market basket analysis, anomaly detection.

Algorithms: K-Means, DBSCAN, Hierarchical Clustering, PCA, t-SNE, Autoencoders, Apriori, Isolation Forest.

3. Semi-Supervised Learning **SEMI-SUPERVISED**

What: Uses a **small amount of labeled data** combined with a **large amount of unlabeled data**.

Why: Labeling data is expensive and time-consuming; this approach gets the best of both worlds.

Examples: Google Photos (face recognition with few labeled faces), medical image classification.

Techniques: Self-training, Co-training, Label Propagation, Generative Models.

▣ 4. Reinforcement Learning **REINFORCEMENT**

What: An **agent** learns by **interacting with an environment**, receiving rewards or penalties for actions.

Goal: Maximize cumulative reward over time through trial and error.

Key Concepts: Agent, Environment, State, Action, Reward, Policy.

Examples: Game playing (AlphaGo, Chess), self-driving cars, robotics, recommendation systems.

Algorithms: Q-Learning, Deep Q-Network (DQN), Policy Gradient, Actor-Critic, PPO.

3

Supervised vs Unsupervised Learning — Deep Comparison

Criteria	Supervised Learning	Unsupervised Learning
Data	Labeled (input + correct output)	Unlabeled (input only)
Goal	Predict outcomes for new data	Discover hidden patterns/structure
Feedback	Direct — compares prediction with actual label	No direct feedback
Complexity	Relatively simpler to evaluate	Harder to evaluate (no ground truth)
Data Requirement	Needs labeled data (expensive to collect)	Works with raw/unlabeled data
Output	Known classes or continuous values	Clusters, components, rules
Evaluation	Accuracy, Precision, Recall, RMSE, R^2	Silhouette Score, Inertia, Visual inspection
Overfitting Risk	Higher (memorize labels)	Lower (no labels to memorize)
Use Cases	Spam detection, price prediction, diagnosis	Customer segmentation, anomaly detection
Algorithms	Linear/Logistic Regression, SVM, RF, KNN	K-Means, DBSCAN, PCA, Apriori

📌 Interview Tip — When to choose which?

Use Supervised when you have labeled historical data and want to predict a specific outcome (e.g., "Will this customer churn? Yes/No").

Use Unsupervised when you want to explore data, find hidden groups, or reduce complexity without a predefined target (e.g., "What types of customers do we have?").

Classification vs Regression — Deep Comparison

Both are **supervised learning** tasks, but they differ in what they predict:

□ Classification

What: Predicts a **discrete/categorical label** — assigns input to one of predefined classes.

Output: A class label (e.g., "Spam" or "Not Spam", "Cat" or "Dog").

Types:

- **Binary Classification:** Two classes (Yes/No, True/False, 0/1). E.g., Fraud detection.
- **Multi-class Classification:** More than two classes. E.g., Digit recognition (0–9).
- **Multi-label Classification:** Each sample can belong to multiple classes. E.g., Tags on an article.

Evaluation Metrics: Accuracy, Precision, Recall, F1-Score, AUC-ROC, Confusion Matrix, Log Loss.

Algorithms: Logistic Regression, Decision Trees, Random Forest, SVM, KNN, Naive Bayes, Neural Networks, XGBoost.

□ Regression

What: Predicts a **continuous numerical value**.

Output: A real number (e.g., \$350,000 house price, 27.5°C temperature).

Types:

- **Simple Linear Regression:** One input feature → one output.
- **Multiple Linear Regression:** Multiple input features → one output.
- **Polynomial Regression:** Non-linear relationships.
- **Ridge/Lasso Regression:** Linear regression with regularization.

Evaluation Metrics: MAE (Mean Absolute Error), MSE (Mean Squared Error), RMSE, R^2 Score, Adjusted R^2 .

Algorithms: Linear Regression, Polynomial Regression, Decision Trees, Random Forest, SVR, Neural Networks, XGBoost Regressor.

Head-to-Head Comparison

Criteria	Classification	Regression
Output Type	Discrete / Categorical	Continuous / Numerical
Question Answered	"What category does this belong to?"	"How much / how many?"
Example Question	Is this email spam or not?	What will the house price be?
Decision Boundary	Separates classes in feature space	Best-fit line/curve through data
Loss Function	Cross-Entropy / Log Loss	MSE / MAE
Last Layer (NN)	Softmax / Sigmoid	Linear activation
Key Metrics	Accuracy, F1, AUC-ROC	R ² , RMSE, MAE

📌 Interview Tip — Tricky Question

Q: "Can Logistic Regression be used for classification even though it has 'Regression' in its name?"

A: Yes! Despite its name, Logistic Regression is a classification algorithm. It uses a sigmoid function to output probabilities (0 to 1), which are then thresholded (usually at 0.5) to assign a class label. The "regression" part refers to the fact that it regresses the log-odds (logit) of the probability.

5

Key ML Algorithms & When to Use Them

1. Linear Regression

Type: Supervised — Regression

How: Finds the best-fit straight line ($Y = mX + b$) that minimizes the sum of squared errors between predicted and actual values.

When to Use: Continuous target variable, linear relationship between features and target, quick baseline model.

Pros: Simple, interpretable, fast. **Cons:** Assumes linearity; sensitive to outliers.

Example: Predicting salary based on years of experience.

2. Logistic Regression

Type: Supervised — Classification (Binary / Multi-class)

How: Uses the **sigmoid function** to squeeze output between 0 and 1 (probability). Applies a threshold to classify.

When to Use: Binary classification problems, when you need probability outputs, baseline for classification tasks.

Pros: Fast, interpretable, gives probabilities. **Cons:** Assumes linear decision boundary.

Example: Email spam detection (Spam = 1, Not Spam = 0).

3. Decision Tree

Type: Supervised — Classification & Regression

How: Splits data into branches based on feature values using criteria like **Gini Impurity** or **Information Gain (Entropy)**. Forms a tree-like structure.

When to Use: Need interpretability, non-linear relationships, both categorical and numerical data.

Pros: Easy to visualize, no feature scaling needed, handles non-linearity.
Cons: Prone to overfitting.

Example: Loan approval decision based on income, age, credit score.

4. Random Forest

Type: Supervised — Classification & Regression (Ensemble)

How: Builds **multiple decision trees** on random subsets of data and features, then aggregates predictions (majority vote for classification, average for regression). This is called **Bagging**.

When to Use: Need robust performance, reduce overfitting of single tree, feature importance ranking.

Pros: High accuracy, handles overfitting, works with large datasets. **Cons:** Less interpretable, slower.

Example: Customer churn prediction.

5. Support Vector Machine (SVM)

Type: Supervised — Classification (primarily) & Regression

How: Finds the **optimal hyperplane** that maximizes the margin between classes. Uses **kernel trick** for non-linear data (RBF, Polynomial kernels).

When to Use: High-dimensional data, clear margin of separation, small to medium datasets, text classification.

Pros: Effective in high dimensions, memory efficient. **Cons:** Slow on large datasets, needs feature scaling.

Example: Image classification, handwriting recognition.

6. K-Nearest Neighbors (KNN)

Type: Supervised — Classification & Regression

How: Classifies a data point based on the **majority vote of its K nearest neighbors** (using distance metrics like Euclidean). It's a **lazy learner** — no explicit training phase.

When to Use: Small datasets, multi-class problems, quick prototyping.

Pros: Simple, no training required, intuitive. **Cons:** Slow at prediction on large data, sensitive to irrelevant features & scale.

Example: Recommender systems, digit recognition.

7. Naive Bayes

Type: Supervised — Classification

How: Based on **Bayes' Theorem** with a "naive" assumption of **feature independence**. Calculates posterior probability for each class.

When to Use: Text classification (NLP), spam filtering, large datasets, real-time prediction.

Pros: Very fast, works well with high-dimensional data, handles missing data.

Cons: Independence assumption is rarely true.

Example: Spam filtering, sentiment analysis.

8. Gradient Boosting (XGBoost / LightGBM / CatBoost)

Type: Supervised — Classification & Regression (Ensemble)

How: Builds trees **sequentially** — each new tree corrects errors of the previous one. Uses gradient descent to minimize the loss. This is called **Boosting**.

When to Use: Kaggle competitions, structured/tabular data, when accuracy matters most.

Pros: State-of-the-art on tabular data, handles missing values, highly customizable. **Cons:** Can overfit, many hyperparameters.

Example: Fraud detection, click-through rate prediction.

9. K-Means Clustering

Type: Unsupervised — Clustering

How: Partitions data into **K clusters** by assigning each point to the nearest centroid, then updating centroids iteratively until convergence.

When to Use: Customer segmentation, grouping similar items, data exploration.

Pros: Simple, scalable. **Cons:** Must choose K, sensitive to initialization, assumes spherical clusters.

Example: Market segmentation by purchase behavior.

10. PCA (Principal Component Analysis)

Type: Unsupervised — Dimensionality Reduction

How: Transforms data into a lower-dimensional space by finding **principal components** (directions of maximum variance).

When to Use: Too many features (curse of dimensionality), visualization of high-dimensional data, preprocessing before ML.

Pros: Reduces noise, speeds up training, aids visualization. **Cons:** Components are hard to interpret, loses some information.

Example: Reducing 100 features to 10 before training a classifier.

Algorithm Selection Cheat Sheet

Scenario	Recommended Algorithm(s)
Predict a number (price, temperature)	Linear Regression, Random Forest Regressor, XGBoost
Yes/No decision	Logistic Regression, Random Forest, XGBoost
Classify images	CNN (Deep Learning), SVM
Text classification / NLP	Naive Bayes, Logistic Regression, Transformers
Group customers	K-Means, DBSCAN, Hierarchical Clustering
Detect anomalies/fraud	Isolation Forest, One-Class SVM, Autoencoders
Too many features	PCA, Feature Selection, Lasso Regression
Small dataset, need accuracy	SVM, KNN

Tabular data competition	XGBoost, LightGBM, CatBoost
Sequential data / time series	LSTM, GRU, ARIMA, Prophet

6 The ML Pipeline — Step by Step

The ML Pipeline is the **end-to-end workflow** for building and deploying a machine learning model. Every ML project follows these stages:



Step 1: Problem Definition 📌

Clearly define **what you want to predict or discover**. Determine:

- Is it a **classification**, **regression**, or **clustering** problem?
- What is the **business objective**? What metric matters most?
- What does **success** look like? (e.g., 95% accuracy, \$10K savings)

Example: "Predict whether a loan applicant will default (binary classification, minimize false negatives)."

Step 2: Data Collection 📌

Gather relevant data from various sources:

- **Internal:** Databases, data warehouses, CSV/Excel files, APIs.
- **External:** Public datasets (Kaggle, UCI), web scraping, third-party APIs.

- **Real-time:** Streaming data (IoT sensors, logs, social media).

Key Considerations: Data quality, quantity, relevance, bias, legal/ethical compliance (GDPR).

Step 3: Data Cleaning & Exploratory Data Analysis (EDA) □

This is often the **most time-consuming step** (60–80% of a data scientist's time).

Data Cleaning:

- **Handle Missing Values:** Drop rows/columns, impute (mean, median, mode, KNN imputation).
- **Remove Duplicates:** Identify and remove duplicate records.
- **Fix Data Types:** Ensure columns have correct types (dates, numbers, categories).
- **Handle Outliers:** Detect using IQR, Z-score; decide to keep, cap, or remove.
- **Fix Inconsistencies:** Standardize formats (e.g., "NY" vs "New York").

EDA (Exploratory Data Analysis):

- **Univariate:** Distribution of each feature (histograms, box plots).
- **Bivariate:** Relationships between features (scatter plots, correlation matrix).
- **Statistical Summary:** Mean, median, std, min, max using `df.describe()`.
- **Visualize class balance** (for classification) — detect imbalanced datasets.

Step 4: Feature Engineering □

Transform raw data into **meaningful features** that improve model performance:

- **Feature Creation:** Create new features (e.g., age from birthdate, BMI from height/weight).
- **Feature Encoding:**
 - **Label Encoding:** Convert categories to numbers (0, 1, 2...). Best for ordinal data.
 - **One-Hot Encoding:** Create binary columns for each category. Best for nominal data.

- **Feature Scaling:**

- **Standardization (Z-score):** Mean = 0, Std = 1. Good for SVM, KNN, Neural Networks.
- **Normalization (Min-Max):** Scale to [0, 1]. Good when you know bounds.

- **Feature Selection:** Remove irrelevant/redundant features (Correlation, Mutual Information, Recursive Feature Elimination).

- **Handling Imbalanced Data:** SMOTE (oversampling minority), undersampling majority, class weights.

Step 5: Data Splitting ✂

Split data into subsets to evaluate model performance on unseen data:

- **Training Set (70-80%):** Used to train/fit the model.
- **Validation Set (10-15%):** Used during training for hyperparameter tuning.
- **Test Set (10-20%):** Used ONLY for final evaluation — never seen during training.

```
X_train, X_test, y_train, y_test = train_test_split(X, y,  
test_size=0.2, random_state=42)
```

Cross-Validation: K-Fold CV splits data into K folds; trains on K-1 folds and validates on the remaining fold, repeated K times. Gives more reliable performance estimate.

Step 6: Model Selection 📊

Choose appropriate algorithms based on:

- **Problem type:** Classification, Regression, or Clustering.
- **Data size:** Small → SVM/KNN; Large → Random Forest/XGBoost/Neural Networks.
- **Interpretability needs:** Need explanation → Decision Tree/Logistic Regression; Accuracy first → Ensemble/DL.
- **Data type:** Tabular → XGBoost; Image → CNN; Text → Transformers; Time Series → LSTM.

Best Practice: Start with a simple baseline model, then iterate with more complex ones.

Step 7: Model Training 🔄

Feed training data to the algorithm so it learns patterns:

- The model adjusts its internal **parameters** (weights, coefficients) to minimize a **loss function**.
- **Gradient Descent**: Iteratively updates parameters in the direction that reduces loss.
- **Epochs**: Number of times the entire training data passes through the model.
- **Batch Size**: Number of samples per gradient update.
- **Learning Rate**: Step size for parameter updates (too high → overshoot; too low → slow).

```
model.fit(X_train, y_train)
```

Step 8: Model Evaluation 📊

Measure model performance on the **test set**:

For Classification:

- **Confusion Matrix**: TP, TN, FP, FN breakdown.
- **Accuracy**: $(TP + TN) / \text{Total}$ — use when classes are balanced.
- **Precision**: $TP / (TP + FP)$ — "Of all predicted positives, how many are actually positive?"
- **Recall (Sensitivity)**: $TP / (TP + FN)$ — "Of all actual positives, how many did we catch?"
- **F1 Score**: Harmonic mean of Precision & Recall — use when classes are imbalanced.
- **AUC-ROC**: Area under the ROC curve — measures overall discriminative ability.

For Regression:

- **MAE**: Average absolute difference between predicted and actual values.
- **MSE**: Average squared difference — penalizes larger errors more.
- **RMSE**: Square root of MSE — same unit as target variable.
- **R² Score**: Proportion of variance explained (1.0 = perfect, 0 = baseline).

Step 9: Hyperparameter Tuning ⚙️

Optimize model settings that are **not learned from data** (set before training):

- **Grid Search:** Exhaustively tries all combinations of specified parameter values.
- **Random Search:** Randomly samples parameter combinations — faster for large search spaces.
- **Bayesian Optimization:** Uses probabilistic model to intelligently pick next set of parameters.
- **Examples of Hyperparameters:** Learning rate, number of trees, max depth, regularization strength, K in KNN.

```
GridSearchCV(estimator, param_grid, cv=5, scoring='accuracy')
```

Step 10: Deployment & Monitoring 📦

Put the model into production and keep it performing well:

- **Serialize the model:** Save using Pickle, Joblib, ONNX, or SavedModel (TF).
- **Create API:** Wrap model in REST API using Flask, FastAPI, or Django.
- **Deploy:** AWS SageMaker, Google Cloud AI, Azure ML, Docker, Kubernetes.
- **Monitor:** Track performance metrics, detect **data drift** and **model drift**.
- **Retrain:** Periodically retrain with new data to maintain accuracy.
- **A/B Testing:** Compare new model against existing one in production.

7

Key ML Terms & Definitions

Quick reference glossary of essential ML terminology:

Feature (X)

An individual measurable property of the data. Also called input variable, predictor, or independent variable. E.g., age, income, height.

Label / Target (Y)

The output variable the model is trying to predict. Also called dependent variable or response. E.g., "Spam/Not Spam", price.

Training Data

The subset of data used to train/fit the model. The model learns patterns from this data.

Test Data

Unseen data used to evaluate model performance after training. Never used during training.

Overfitting

Model learns noise/details of training data too well, performing great on training but poorly on new data. High variance, low bias.

Underfitting

Model is too simple to capture underlying patterns. Performs poorly on both training and test data. High bias, low variance.

Bias

Error due to oversimplified assumptions. High bias → underfitting. E.g., fitting a linear model to non-linear data.

Variance

Error due to model's sensitivity to training data fluctuations. High variance → overfitting. Model changes significantly with different training sets.

Bias-Variance Tradeoff

Finding the sweet spot: decrease bias → increase variance (and vice versa). Goal: minimize total error = $\text{bias}^2 + \text{variance} + \text{noise}$.

Regularization

Technique to prevent overfitting by adding a penalty for complexity. L1 (Lasso): can zero out features. L2 (Ridge): shrinks coefficients.

Loss / Cost Function

Measures how wrong the model's predictions are. The model's goal is to minimize this. E.g., MSE for regression, Cross-Entropy for classification.

Gradient Descent

Optimization algorithm that iteratively adjusts parameters to minimize the loss function by following the negative gradient.

Learning Rate**Epoch**

Controls the step size during gradient descent. Too high → diverge/overshoot. Too low → very slow convergence.

One complete pass through the entire training dataset during model training.

Batch Size

Number of training samples used in one iteration of gradient descent. Mini-batch (32, 64, 128) is most common.

Hyperparameters

Settings configured before training (not learned). E.g., learning rate, number of trees, K in KNN. Tuned via Grid/Random Search.

Parameters

Values learned during training. E.g., weights in linear regression, split points in decision trees.

Cross-Validation

Technique to assess model generalization. K-Fold: split data into K parts, train on K-1, test on 1, repeat K times.

Confusion Matrix

Table showing TP, TN, FP, FN for classification. Foundation for calculating Accuracy, Precision, Recall, F1.

Precision vs Recall

Precision = $TP / (TP + FP)$: "When I say positive, am I right?" Recall = $TP / (TP + FN)$: "Did I find all positives?"

Ensemble Learning

Combining multiple models for better performance. Bagging (Random Forest), Boosting (XGBoost), Stacking.

Feature Scaling

Normalizing feature ranges so no single feature dominates. Required for SVM, KNN, Neural Networks, Gradient Descent.

Curse of Dimensionality

As features increase, data becomes sparse, distances lose meaning, and models need exponentially more data to generalize.

Data Drift

When the distribution of incoming data changes over time compared to training data, causing model performance to degrade.

One-Hot Encoding

Converting categorical variables into binary columns. "Color: [Red, Blue, Green]" → 3 binary columns.

Label Encoding

Converting categories to integers. "Small=0, Medium=1, Large=2". Use for ordinal data only.

SMOTE

Synthetic Minority Over-sampling Technique. Creates synthetic samples

Kernel Trick

for minority class to handle imbalanced datasets.

Used in SVM to transform non-linearly separable data into higher dimensions where a linear separator exists.

Common Interview Questions & Model Answers

Q1: What is the difference between supervised and unsupervised learning?

Answer: Supervised learning uses labeled data where the model learns a mapping from inputs to known outputs (e.g., predicting house prices). Unsupervised learning uses unlabeled data where the model discovers hidden patterns or groupings on its own (e.g., customer segmentation). The key difference is the presence of a target variable — supervised has one, unsupervised does not.

Q2: What is overfitting, and how do you prevent it?

Answer: Overfitting occurs when a model learns the noise and details of training data so well that it fails to generalize to new, unseen data. Signs: high training accuracy but low test accuracy.

Prevention techniques:

- Use more training data
- Apply regularization (L1/L2)
- Use cross-validation
- Reduce model complexity (fewer features, shallower trees)
- Early stopping (in neural networks)
- Use ensemble methods (Random Forest, Bagging)
- Dropout (in neural networks)

Q3: Explain the Bias-Variance Tradeoff

Answer: Bias is error from wrong assumptions (underfitting); Variance is error from sensitivity to training data (overfitting). The tradeoff means reducing one often increases the other. A simple model has high bias, low variance. A complex model has low bias, high variance. The goal is to find the sweet spot that minimizes total error. For example, a linear model on non-linear data has high bias; a degree-20 polynomial has high variance.

Q4: When would you use Random Forest over XGBoost?

Answer: Use **Random Forest** when you want simplicity, faster training with fewer hyperparameters to tune, and when overfitting is a concern (bagging reduces variance). Use **XGBoost** when you need maximum predictive accuracy on tabular data, can afford time for hyperparameter tuning, and need features like built-in handling of missing values. XGBoost tends to perform better in competitions, but Random Forest is more robust out-of-the-box.

Q5: How do you handle imbalanced datasets?

Answer:

- **Resampling:** Oversample minority (SMOTE) or undersample majority class.
- **Class Weights:** Assign higher weight to minority class in the loss function.
- **Use appropriate metrics:** Don't rely on accuracy; use F1-Score, Precision-Recall AUC, or Cohen's Kappa.
- **Ensemble methods:** Balanced Random Forest, EasyEnsemble.
- **Collect more data** for the minority class if possible.
- **Anomaly detection** approach if the minority class is extremely rare.

Q6: What is Cross-Validation and why is it important?

Answer: Cross-validation is a technique to assess how well a model generalizes to independent data. In K-Fold CV, the dataset is split into K equal parts. The model is trained K times, each time using K-1 folds for training and 1 fold for validation. The final score is the average across all folds. It's important because: (1) It gives a more reliable performance estimate than a single train-test split, (2) It reduces the chance that results are due to a lucky/unlucky split, (3) It uses all data for both training and validation.

Q7: Explain the difference between Parameters and Hyperparameters

Answer: **Parameters** are learned from data during training (e.g., weights in linear regression, split thresholds in decision trees). **Hyperparameters** are set before training and control the learning process (e.g., learning rate, number of trees, max depth, K in KNN). Parameters are optimized by the

algorithm itself; Hyperparameters are optimized by the data scientist using techniques like Grid Search, Random Search, or Bayesian Optimization.

Q8: What is the purpose of a Confusion Matrix?

Answer: A confusion matrix is a table that summarizes the performance of a classification model by showing the counts of True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). It helps you understand not just overall accuracy, but where the model is making mistakes. From it, you can calculate Precision, Recall, F1-Score, and Specificity — which are critical when dealing with imbalanced datasets or when different types of errors have different costs.

Q9: When do you need feature scaling and when don't you?

Answer:

Need scaling: Algorithms that use distance measures or gradient-based optimization: KNN, SVM, Neural Networks, Logistic Regression, Linear Regression, K-Means, PCA.

Don't need scaling: Tree-based algorithms (Decision Trees, Random Forest, XGBoost) because they split based on thresholds, not distances. Naive Bayes also doesn't require scaling.

Q10: What is the difference between Bagging and Boosting?

Answer: Both are ensemble techniques that combine multiple models.

Bagging (Bootstrap Aggregating): Trains multiple models independently on random subsets (with replacement) and aggregates their predictions (voting/averaging). Reduces **variance**. Example: Random Forest.

Boosting: Trains models sequentially, where each new model focuses on correcting errors of the previous one. Reduces **bias**. Example: XGBoost, AdaBoost, LightGBM.

Key difference: Bagging → parallel, reduces variance. Boosting → sequential, reduces bias.



Quick Formula Reference Sheet

Metric	Formula	When to Use
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	Balanced classes only
Precision	$TP / (TP + FP)$	When FP is costly (spam, fraud alerts)
Recall	$TP / (TP + FN)$	When FN is costly (disease, fraud miss)
F1 Score	$2 \times (Precision \times Recall) / (Precision + Recall)$	Imbalanced classes
MAE	$(1/n) \times \sum y_i - \hat{y}_i $	Regression, easy to interpret
MSE	$(1/n) \times \sum (y_i - \hat{y}_i)^2$	Regression, penalizes large errors
RMSE	$\sqrt{MSE}$	Regression, same unit as target
R² Score	$1 - (SS_{res} / SS_{tot})$	Regression, proportion of variance explained

Final Interview Tips

- **Always start simple:** Mention starting with a baseline model before trying complex ones.
- **Think out loud:** Explain your reasoning when choosing an algorithm or metric.
- **Ask clarifying questions:** "Is the dataset balanced? How big is it? What's the cost of a false negative?"
- **Know the tradeoffs:** Every algorithm has pros and cons — show you understand when to use what.
- **Mention the full pipeline:** Show you think end-to-end, not just about the model.
- **Be honest:** If you don't know something, say "I'd need to look that up" rather than guessing.

